

补充：wc、sort、uniq 使用详解

概述

wc、sort、uniq 是文本处理的三个重要辅助工具，它们经常与三剑客配合使用，形成强大的数据处理流水线。

工具	名称含义	核心功能
wc	Word Count	统计行数、单词数、字符数
sort	排序	对文本行进行排序
uniq	Unique	去除或统计重复行

一、wc (Word Count)

1.1 基本功能

wc 用于统计文件的行数、单词数和字节数。

基本语法：

```
wc [选项] 文件
```

1.2 常用选项

选项	作用	示例
<code>-l</code>	只统计行数	<code>wc -l file</code>
<code>-w</code>	只统计单词数	<code>wc -w file</code>
<code>-c</code>	只统计字节数	<code>wc -c file</code>
<code>-m</code>	只统计字符数（支持多字节字符）	<code>wc -m file</code>

1.3 wc 实战练习

```
# 准备测试文件
cat > /tmp/wc_test.txt << 'EOF'
Hello World
This is a test file
It has multiple lines
And some words
你好世界
EOF

# 显示完整统计（行数 单词数 字节数 文件名）
wc /tmp/wc_test.txt
# 输出: 5  14  89 /tmp/wc_test.txt

# 只统计行数
wc -l /tmp/wc_test.txt
# 输出: 5 /tmp/wc_test.txt

# 只获取数字（去掉文件名）
wc -l < /tmp/wc_test.txt
# 输出: 5

# 统计多个文件
wc -l /etc/passwd /etc/group
# 会显示每个文件的行数和总计
```

```
# 统计单词数
wc -w /tmp/wc_test.txt

# 统计字符数（注意中文字符）
wc -m /tmp/wc_test.txt
```

1.4 wc 常用场景

```
# 场景 1: 统计代码行数
find /path/to/project -name "*.py" | xargs wc -l

# 场景 2: 统计目录下文件数量
ls -l /etc | wc -l

# 场景 3: 统计进程数量
ps aux | wc -l

# 场景 4: 统计日志中错误数量
grep -c 'ERROR' /var/log/app.log
# 或
grep 'ERROR' /var/log/app.log | wc -l

# 场景 5: 统计用户数量
wc -l < /etc/passwd

# 场景 6: 统计空行数量
grep -c '^$' /tmp/wc_test.txt

# 场景 7: 统计非空行数量
grep -v '^$' /tmp/wc_test.txt | wc -l
```

二、sort（排序）

2.1 基本功能

sort 用于对文本行进行排序，默认按 ASCII 码顺序升序排列。

基本语法：

```
sort [选项] 文件
```

2.2 常用选项

选项	作用	示例
-n	按数值排序	<code>sort -n file</code>
-r	降序（逆序）排序	<code>sort -r file</code>
-k	按指定列排序	<code>sort -k2 file</code>
-t	指定字段分隔符	<code>sort -t: -k3 file</code>
-u	排序并去重	<code>sort -u file</code>
-f	忽略大小写	<code>sort -f file</code>
-h	按人类可读数值排序（K, M, G）	<code>sort -h file</code>
-o	输出到指定文件	<code>sort file -o sorted_file</code>
-c	检查是否已排序	<code>sort -c file</code>
-s	稳定排序	<code>sort -s file</code>
-M	按月份排序	<code>sort -M file</code>

2.3 sort 实战练习

准备测试数据

```
# 创建数字测试文件
cat > /tmp/numbers.txt << 'EOF'
```

```
10
2
100
25
3
1
EOF

# 创建成绩表
cat > /tmp/scores.txt << 'EOF'
Alice 90 85 92
Bob 78 95 88
Charlie 88 76 90
David 92 88 85
Eve 85 90 95
EOF

# 创建带分隔符的数据
cat > /tmp/data.csv << 'EOF'
name,age,salary
John,28,50000
Alice,35,75000
Bob,22,45000
Charlie,45,90000
David,30,60000
EOF
```

练习 1：基本排序

```
# 默认排序（按字符串）
sort /tmp/numbers.txt
# 输出: 1, 10, 100, 2, 25, 3 (按字符排序, 10 在 2 前面)

# 按数值排序
sort -n /tmp/numbers.txt
# 输出: 1, 2, 3, 10, 25, 100

# 降序排序
sort -nr /tmp/numbers.txt
# 输出: 100, 25, 10, 3, 2, 1
```

练习 2：实际应用场景

```
# 场景 1: 找出占用空间最大的目录
du -sh /var/* 2>/dev/null | sort -hr | head -10

# 场景 2: 按 CPU 使用率排序进程
ps aux | sort -k3 -nr | head -10

# 场景 3: 按内存使用率排序进程
ps aux | sort -k4 -nr | head -10

# 场景 4: 对 IP 地址排序
echo -e "192.168.1.10\n192.168.1.2\n192.168.1.100\n10.0.0.1" | sort -t. -k1,1n -k2,2n -k3,3n -k4,4n

# 场景 5: 排序并保存到原文件
sort -n /tmp/numbers.txt -o /tmp/numbers.txt
```

2.4 sort -k 详解

`-k` 选项的完整格式是 `-k start,end`，可以精确指定排序的起止位置：

```
# -k2          从第2列到行尾
# -k2,2        只看第2列
# -k2,3        从第2列到第3列
# -k2.3,2.5    第2列的第3到第5个字符
```

```
# 示例：按第2列数值排序
```

```
sort -k2,2n /tmp/scores.txt
```

```
# 示例：先按第2列排序，相同时按第1列排序
```

```
sort -k2,2n -k1,1 /tmp/scores.txt
```

三、uniq（去重/统计重复）

3.1 基本功能

uniq 用于去除或统计**相邻**的重复行。重要特点是只处理相邻的重复行，所以通常需要先 sort 排序。

基本语法：

```
uniq [选项] 文件
```

3.2 常用选项

选项	作用	示例
<code>-c</code>	统计重复次数	<code>uniq -c file</code>
<code>-d</code>	只显示重复的行	<code>uniq -d file</code>
<code>-u</code>	只显示不重复的行	<code>uniq -u file</code>
<code>-i</code>	忽略大小写	<code>uniq -i file</code>
<code>-f N</code>	跳过前 N 个字段	<code>uniq -f 1 file</code>
<code>-s N</code>	跳过前 N 个字符	<code>uniq -s 3 file</code>
<code>-w N</code>	只比较前 N 个字符	<code>uniq -w 5 file</code>

3.3 uniq 实战练习

准备测试数据

```
# 创建有重复行的文件
cat > /tmp/fruits.txt << 'EOF'
apple
apple
banana
apple
cherry
cherry
cherry
banana
date
EOF

# 创建日志模拟
cat > /tmp/ip_log.txt << 'EOF'
192.168.1.1
192.168.1.2
192.168.1.1
192.168.1.1
```



```
192.168.1.3
192.168.1.2
192.168.1.1
EOF
```

练习 1：基本去重

```
# 直接使用 uniq (只去除相邻重复)
uniq /tmp/fruits.txt
# 注意：第二个 apple 没有被去除，因为它不相邻

# 先排序再去重 (正确做法)
sort /tmp/fruits.txt | uniq

# 或者用 sort -u (排序并去重)
sort -u /tmp/fruits.txt
```

练习 2：统计重复次数

```
# 统计每个词出现的次数
sort /tmp/fruits.txt | uniq -c
# 输出：
#      3 apple
#      2 banana
#      3 cherry
#      1 date

# 按次数排序 (常用组合)
sort /tmp/fruits.txt | uniq -c | sort -rn
# 输出：
#      3 cherry
#      3 apple
#      2 banana
#      1 date

# 统计 IP 访问次数并排序
sort /tmp/ip_log.txt | uniq -c | sort -rn
```

练习 3：只显示重复或不重复的行

```
# 只显示重复的行（出现超过1次）
sort /tmp/fruits.txt | uniq -d
# 输出: apple, banana, cherry

# 只显示不重复的行（只出现1次）
sort /tmp/fruits.txt | uniq -u
# 输出: date

# 统计重复行的次数
sort /tmp/fruits.txt | uniq -dc
```

练习 4：忽略大小写

```
# 创建测试数据
echo -e "Apple\napple\nAPPLE\nBanana\nbanana" > /tmp/case_test.txt

# 忽略大小写去重
sort -f /tmp/case_test.txt | uniq -i

# 忽略大小写统计
sort -f /tmp/case_test.txt | uniq -ic
```

四、组合使用（sort + uniq + wc）

这三个命令经常组合使用，形成经典的统计模式：

4.1 经典模式：排序 → 去重统计 → 再排序

```
# 模式: xxx | sort | uniq -c | sort -rn
# 用途: 统计出现次数并按次数排序

# 示例 1: 统计日志中各 IP 的访问次数 (Top 10)
awk '{print $1}' /tmp/access.log | sort | uniq -c | sort -rn | head -10

# 示例 2: 统计最常用的命令
history | awk '{print $2}' | sort | uniq -c | sort -rn | head -10

# 示例 3: 统计文件类型分布
find /etc -type f | sed 's/.*\.//' | sort | uniq -c | sort -rn | head -10
```

4.2 实战场景

```
# 场景 1: 分析 Nginx 访问日志
# 创建模拟日志
cat > /tmp/nginx.log << 'EOF'
192.168.1.1 - - [01/Jan/2024:10:00:01] "GET /index.html" 200
192.168.1.2 - - [01/Jan/2024:10:00:02] "GET /about.html" 200
192.168.1.1 - - [01/Jan/2024:10:00:03] "GET /index.html" 200
192.168.1.3 - - [01/Jan/2024:10:00:04] "POST /api/login" 200
192.168.1.1 - - [01/Jan/2024:10:00:05] "GET /contact.html" 404
192.168.1.2 - - [01/Jan/2024:10:00:06] "GET /index.html" 200
192.168.1.1 - - [01/Jan/2024:10:00:07] "GET /index.html" 200
192.168.1.4 - - [01/Jan/2024:10:00:08] "GET /admin" 403
EOF

# 统计访问量最高的 IP
awk '{print $1}' /tmp/nginx.log | sort | uniq -c | sort -rn

# 统计访问量最高的页面
awk '{print $7}' /tmp/nginx.log | sort | uniq -c | sort -rn

# 统计各状态码数量
awk '{print $9}' /tmp/nginx.log | sort | uniq -c | sort -rn

# 统计总访问量
```

```
wc -l < /tmp/nginx.log
```

```
# 统计独立 IP 数量
```

```
awk '{print $1}' /tmp/nginx.log | sort -u | wc -l
```

```
# 场景 2：分析系统用户
```

```
# 统计各 shell 的用户数量
```

```
awk -F: '{print $NF}' /etc/passwd | sort | uniq -c | sort -rn
```

```
# 统计各 GID 的用户数量
```

```
awk -F: '{print $4}' /etc/passwd | sort | uniq -c | sort -rn
```

```
# 场景 3：代码分析
```

```
# 统计代码行数（排除空行和注释）
```

```
grep -v '^$' *.py 2>/dev/null | grep -v '^#' | wc -l
```

```
# 统计函数定义数量
```

```
grep -h '^def ' *.py 2>/dev/null | wc -l
```

```
# 找出重复的函数名
```

```
grep -h '^def ' *.py 2>/dev/null | awk '{print $2}' | sed 's/(.*//g' | sort | uniq -d
```

```
# 场景 4：网络分析
```

```
# 统计当前连接数最多的 IP
```

```
netstat -an | grep ESTABLISHED | awk '{print $5}' | cut -d: -f1 | sort | uniq -c | sort -rn | head -10
```

```
# 统计各连接状态的数量
```

```
netstat -an | awk '/^tcp/ {print $6}' | sort | uniq -c | sort -rn
```

4.3 复杂组合示例

```
# 示例 1：统计每小时的访问量
```

```
awk -F'[:\\\[ ]' '{print $2}' /tmp/nginx.log | sort | uniq -c
```

```
# 示例 2：找出只访问过一次的 IP
```

```
awk '{print $1}' /tmp/nginx.log | sort | uniq -c | awk '$1==1 {print $2}'
```

示例 3: 找出访问超过 3 次的 IP

```
awk '{print $1}' /tmp/nginx.log | sort | uniq -c | awk '$1>3 {print $2, $1}'
```

示例 4: 统计每个 IP 访问了多少个不同的页面

```
awk '{print $1, $7}' /tmp/nginx.log | sort -u | awk '{ip[$1]++} END {for(i in ip) print i, ip[i]}'
```

示例 5: 综合报表

```
echo "==== 访问统计报表 ====="
echo ""
echo "总访问量: $(wc -l < /tmp/nginx.log)"
echo "独立 IP 数: $(awk '{print $1}' /tmp/nginx.log | sort -u | wc -l)"
echo ""
echo "Top 5 IP:"
awk '{print $1}' /tmp/nginx.log | sort | uniq -c | sort -rn | head -5
echo ""
echo "Top 5 页面:"
awk '{print $7}' /tmp/nginx.log | sort | uniq -c | sort -rn | head -5
echo ""
echo "状态码分布:"
awk '{print $9}' /tmp/nginx.log | sort | uniq -c | sort -rn
```

五、命令对比速查表

功能需求	推荐命令	示例
统计文件行数	<code>wc -l</code>	<code>wc -l file</code>
统计单词数	<code>wc -w</code>	<code>wc -w file</code>
统计字节/字符数	<code>wc -c</code> / <code>wc -m</code>	<code>wc -c file</code>
按字母顺序排序	<code>sort</code>	<code>sort file</code>
按数值排序	<code>sort -n</code>	<code>sort -n file</code>
降序排序	<code>sort -r</code>	<code>sort -rn file</code>
去除相邻重复行	<code>uniq</code>	<code>sort file uniq</code>
统计重复次数	<code>uniq -c</code>	<code>sort file uniq -c</code>
只显示重复行	<code>uniq -d</code>	<code>sort file uniq -d</code>
只显示唯一行	<code>uniq -u</code>	<code>sort file uniq -u</code>

六、经典组合速记

```
# 1. 统计次数并排序（最常用）
xxx | sort | uniq -c | sort -rn

# 2. 统计独立项数量
xxx | sort -u | wc -l

# 3. 找出 Top N
xxx | sort | uniq -c | sort -rn | head -N
```

4. 找出重复项

```
xxx | sort | uniq -d
```

5. 找出唯一项

```
xxx | sort | uniq -u
```

6. 找出出现超过 N 次的项

```
xxx | sort | uniq -c | awk '$1>N {print}'
```